Autonomous Drift Parking with Lidar and Nonlinear MPC on a 1:10 Ackermann Car

Pietro Gelmini
University of Trento, Italy
gelmini.pietro@gmail.com

Abstract—A 1:10 Ackermann car with hard plastic tires parks itself sideways between two boxes on a tiled floor, drifting into gaps as short as 47 cm, 1.8 times its length. Its dynamics were identified from an overhead camera, an IMU and the wheel encoders: a single-track model with saturating tires and a spool rear axle. On it a nonlinear model predictive controller (NMPC, acados) runs on the car's Raspberry Pi 4 at 50 Hz and holds a tight line where Pure Pursuit cuts the corners, or spins on a slippery floor in simulation. A first goal, holding a sustained drift around a cone, worked in simulation and failed on the car: the drift the controller held is an unstable equilibrium of the model that the real car does not keep. The final manoeuvre uses the regime where the model is right, the slide with the rear wheels locked: the lidar measures the gap, the car kicks into a drift at full lock, and the brake is timed on the gyro so that the slide ends at 180°. Over nine runs the car stopped at $(180.8 \pm 2.9)°$. It also parks between boxes placed anywhere in a mapped room, planning and driving the approach with the NMPC, and runs a one-take gymkhana (slalom, donut, parking), ten clean takes on the day it was filmed. The same software runs unchanged in a Gazebo twin of the car and the room.

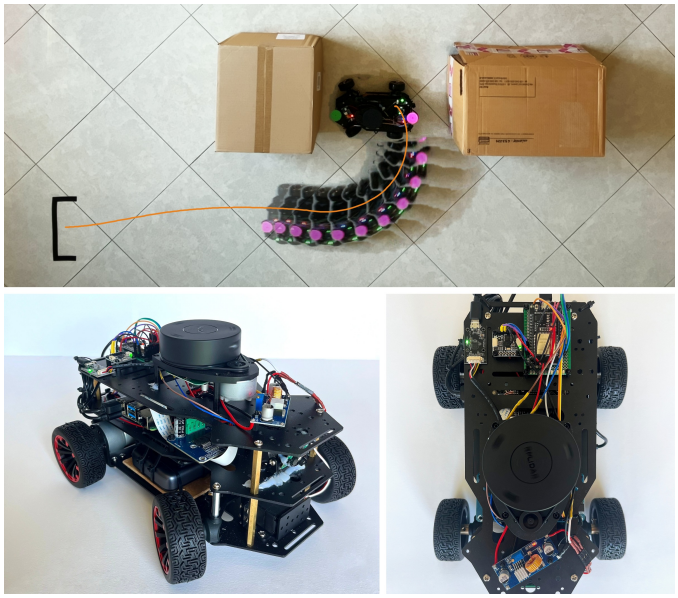


Fig. 1. Top: the car parking itself into a 47 cm gap, seen by an overhead camera; the car every 0.1 s from the kick to the stop, and the path of its rear axle from the start mark (left). Bottom: the car, 0.26 m long and 2.0 kg, with the lidar on top.

I. Introduction

Driving at the limit of tire friction is where vehicle control is both hardest and most needed: emergency manoeuvres on ice, racing, recovery from a slide. Full-scale research platforms have shown controlled drifting along general paths [1], [2] and autonomous sliding into a parking spot [3]. Small-scale cars make such experiments cheap and safe [4]–[6], but they move the difficulty elsewhere: tires that are hard and poorly characterized, low-rate sensors, and all computation on a single-board computer.

This report describes a complete stack on a car built for about 510 euros, from the identification of its dynamics to autonomous manoeuvres beyond the limit of grip, including one that did not work. The contributions are:

- an identified dynamic model of a 1:10 car with a spool rear axle and saturating tires, from overhead video, IMU and encoders, refitted for locked-wheel slides on lidar ground truth (Sec. III);
- an NMPC on that model running on the car's Raspberry Pi 4 at 50 Hz, tested against Pure Pursuit on the car and, at lower friction, in simulation (Sec. IV);
- a negative result on sustained drifting: the drift that the controller holds in simulation is an unstable steady state of the model that the real car does not have (Sec. V);
- drift parking: sideways into a lidar-measured gap of 1.8 car lengths, between boxes placed anywhere in a room, and a one-take gymkhana (Sec. VI, VII);
- a Gazebo twin running the identical software stack (Sec. VIII).

Code, notebooks with every analysis, and the video are at https://github.com/Gelminaio/drift-mpc-ackermann.

II. Platform

The car (Fig. 1) is a 1:10 Ackermann chassis with servo steering and a rear axle driven as a spool by two geared DC motors with quadrature encoders (660 ticks per revolution) through BTS7960 half bridges. It weighs 2.015 kg, 57 % on the rear axle, with a wheelbase of 0.173 m. Hard plastic drift tires on tiles slide at a friction coefficient of about 0.25–0.28 at the rear. The steering reaches 28.9° to the left and 21.8° to the right; the motors limit the wheel speed to 1.2 m/s.

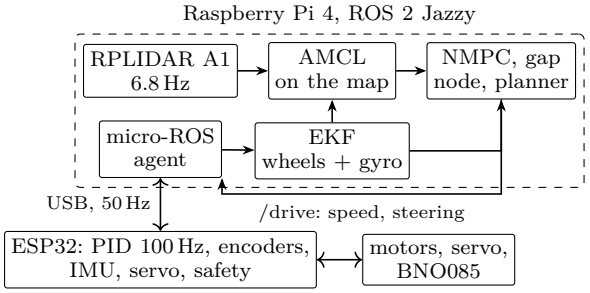


Fig. 2. Hardware and software. Everything runs on the car; the same nodes run in Gazebo against a plugin that replaces the ESP32 and the car.

Two computers share the work (Fig. 2). An ESP32 runs the real-time layer under FreeRTOS: wheel speed PID control at 100 Hz, encoders, a BNO085 IMU, the servo, and a safety supervisor (arming, command timeout, stall detection, watchdog). It talks micro-ROS over USB serial and stamps its messages on the clock of the Raspberry Pi 4, where they arrive 7 ms old. A speed setpoint of zero shorts the motor: the rear wheels lock within 0.35–0.4 s, which is the handbrake of Sec. VI. The Pi runs ROS 2 Jazzy [7]: the RPLIDAR A1 driver (6.8 Hz), wheel odometry, an EKF on wheel speed and gyro (robot_localization [8]), AMCL [9] on an occupancy map of the room (first from slam_toolbox [10], later rebuilt from registered scans), and the controllers. A desktop runs only the simulation and the analysis. Measured against the encoder count, the lidar scan is 56–76 ms old when it arrives and the gyro 44–64 ms; the controllers carry the state forward over these delays.

III. Model and identification

A. Single-track model

The body velocities v_x, v_y and yaw rate r are those of the centre of gravity (CG), with l_f and l_r its distances to the axles and T the track. The front axle is one tire at slip angle $\alpha_f = \delta - \arctan\big((v_y + l_f r)/v_x\big)$,

$$F_{yf} = \mu_f N_f \sin\big(C \arctan(B_f \alpha_f)\big), \tag{1}$$

a simplified magic formula [11]. The rear is a spool: both wheels turn at the same surface speed u and each slides over the ground at its own velocity $(v_{x,i}, v_{y,i}) = (v_x \mp \frac{T}{2} r,\ v_y - l_r r)$. With the slip vector $\boldsymbol{\sigma}_i = (v_{x,i} - u,\ v_{y,i})/u$, each rear tire pushes against its own sliding direction,

$$\mathbf{F}_i = -\frac{\mu_r N_r}{2} \sin\big(C \arctan(B_r |\boldsymbol{\sigma}_i|)\big) \frac{\boldsymbol{\sigma}_i}{|\boldsymbol{\sigma}_i|}, \tag{2}$$

sideways in a turn, forward when the wheels spin, backward when they are locked. The steering angle follows its command through a first-order lag τ_δ and the wheel speed follows its setpoint through the motor control loop. A floor with a different friction is a scale k on the peak forces, $\mu \to k\mu$ and $B \to B/k$, which keeps the cornering stiffness: the brush model's behaviour [11].

TABLE I
Identified parameters

mass; yaw inertia	m; I_z	2.015 kg; 0.0125 kg m^2
CG to the axles; height	l_f, l_r; h	98.6, 74.4; 67 mm
track; wheel radius	T; R_w	174.5; 29.9 mm
front tire	μ_f, B_f	0.449, 60
rear tire	μ_r, B_r	0.315, 20.8
shape factor	C	0.678
full slide, front; rear	$\mu \sin(C\pi/2)$	0.393; 0.276
steering lag; servo rate	τ_δ	0.169 s; 250 °/s
wheel speed lag; top speed		15–45 ms; 1.2 m/s

B. Identification

Geometry and mass were measured directly, the CG from the axle loads on a level and a tilted car, the yaw inertia with a bifilar pendulum [12]: $I_z = 0.0125\,\text{kg m}^2 \pm 2.3\,\%$. For the dynamics, a phone 2 m above the floor filmed the car with two coloured markers 267 mm apart on its centre line; the markers give position, heading and, with the encoders, the CG velocity and sideslip, aligned with the bag on the yaw rate. Steering steps, a steering chirp (0.2–2.5 Hz), a drift entry and exit, and a steering release inside the drift were each driven twice, one run to fit and one to validate. The five tire parameters and τ_δ were fitted by least squares on yaw rate and sideslip over the three identification runs together, so that one parameter set covers grip and drift (Table I).

The fit has $C < 1$: the tire curves rise to $\mu N \sin(C\pi/2)$ and stay there, with no peak. At full slide the rear gives 0.276, against 0.248 from straight launches with wheelspin, an independent test. The steering lag is 0.17 s, against 25–40 ms for the servo alone on a stand: most of it lies between the wheel angle and the tire force. Predicting 1 s ahead from the measured state, the way the NMPC uses the model, the yaw rate is within an NRMSE of 11–17% on the validation runs, the sideslip within 1.4–1.6° in grip and 3.1–3.9° in the drift (Fig. 3).

For the locked-wheel slides of Sec. VI the model was refitted on a second ground truth: the path of the rear axle from the lidar, every scan registered by ICP [13] on the scans taken at rest, each beam moved to where the car was when it was taken (0.9–1.3 cm rms per scan). The refit adds lateral and longitudinal load transfer and, while braked, separate rear friction factors along and across the wheel; it asks for a locked rear that slides with 1.37 times the friction along the wheel and 0.94 times across it. Over held-out windows of 1.2 s the refit ends 2.4–6.2 cm off, against 4.5–7.2 cm for the first model.

IV. Nonlinear MPC at the limit

A. Problem

The NMPC tracks a path with the rear axle rather than the CG. In grip the rear axle moves along the car's heading, so a heading error of zero means the car is aligned with the path; tracking the CG instead, which slides by

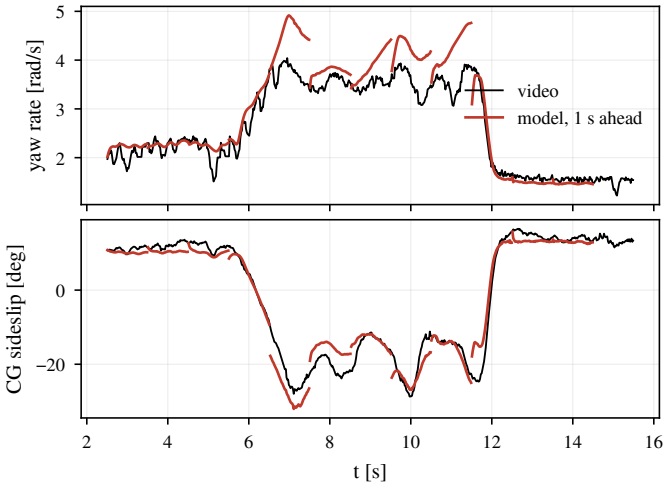


Fig. 3. The model 1 s ahead from the measured state, restarted every second, against the overhead video on the validation drift run: cornering at 0.9 m/s, the drift at full throttle, back to grip.

5–7° in a corner, made the controller lock the rear wheels to rotate the car. The state is

$$x = (s,\ n,\ e_\psi,\ v_x,\ v_y,\ r,\ \delta,\ u,\ \delta_c,\ u_c), \tag{3}$$

progress s, lateral offset n and heading error e_ψ of the rear axle to the path, the body states, the steering angle and wheel speed, and the two commands sent to the car. The inputs are the command rates, bounded by the servo (4.36 rad/s) and by 5 m/s^2; the commands are bounded by the steering map and 0–1.2 m/s. With path curvature $\kappa(s)$ and the lateral velocity of the rear axle $v_{y,r} = v_y - l_r r$,

$$\dot{s} = \frac{v_x \cos e_\psi - v_{y,r} \sin e_\psi}{1 - n\kappa}, \quad \dot{n} = v_x \sin e_\psi + v_{y,r} \cos e_\psi,$$
$$\dot{e}_\psi = r - \kappa \dot{s}, \tag{4}$$

with the body dynamics of Sec. III. The cost is a weighted sum of n (5 cm), e_ψ (0.1 rad), the speed error against a profile (0.3 m/s) and the command rates; $|n| < 0.3$ m and $v_x > 0.3$ m/s are soft constraints. There is no bound on the tire slip: the tires may saturate.

The horizon is 1 s in 25 stages, 10 of 20 ms and 15 of 53 ms. acados [14] generates the solver from the CasADi [15] model: an implicit Gauss–Legendre integrator [16] (the front tire makes the model stiff at low speed, where explicit RK4 needs steps under 4 ms), one Gauss–Newton SQP iteration per control step in the spirit of the real-time iteration [17], HPIPM [18] for the QP, and a merit-function line search. The plain full step overshoots where the tires saturate: the steering chattered on 25–38% of the steps. A 50 Hz horizon of 50 stages took 10.5 ms on the Pi; the split horizon takes 5.6 ms on average and 9.6 ms at worst with the whole stack running. The C++ node publishes a command every 20 ms; on the car it solved in 8 ms on average and exceeded the step once in 1548.

TABLE II
Rear axle off the line, rms (max) [cm]: Python closed loop on the true state, Gazebo with the whole stack against the true path, the car against its localization

	floor; line	Pure Pursuit	NMPC
Python	tiles	3.2 (6.8)	1.6 (3.1)
	25% grip, profile at 120%	spins	1.8 (4.5)
Gazebo	35% grip	7.6 (28)	3.3 (8)
	25% grip	spins	6.7 (18)
car	tiles; 0.4 m corners	7.1 (19.4)	2.7 (7.4)
	tiles; 0.6 m corners	3.4 (8.5)	3.5 (7.1)

B. Results

Table II compares the NMPC with Pure Pursuit [19], tuned on the same model with a delayed pose. In a Python closed loop on the true state the NMPC halves the error and holds the line at 120% of the grip a speed profile assumes, where Pure Pursuit spins. In Gazebo with the full stack and the friction scaled, Pure Pursuit spins in the first lap at 25% of the grip of the tiles while the NMPC drives three laps, 10 % slower (Fig. 4). On the car, on a line with 0.4 m corners driven at full speed, the NMPC stays 2.7 cm rms from it against 7.1 cm: Pure Pursuit cuts the corners, its lookahead being longer than their arcs. On a wider line the two are even (3.5 cm against 3.4 cm), and there the error is the localization's: about 6 cm rms against the ground truth in Gazebo, and the NMPC, which steers twice as much as Pure Pursuit, chases it.

V. Drift around a cone: a negative result

With the steering and the wheel speed held, the model has up to three steady states [20]: grip, a saddle, and a drift. The drift is stable on its own because the firmware holds the wheel speed, and with it the rear slip; it matches the donut of the identification runs (0.49 m/s, −18° of sideslip at the CG against the measured 0.50 m/s and −18° to −21°). These stable drifts are tight, 11–19 cm of CG radius, too tight for a cone at the centre. The goal of a drift around a cone was a wider, unstable equilibrium (13° of steering, 1.12 m/s at the wheels, 0.27 m of radius and −25° of rear slip) held by an NMPC with the same model and solver, its reference the cone in the lidar scan.

In Gazebo it worked: the rear axle held (28.8 ± 0.1) cm from the cone against 28.9 cm, within 1.1 cm with the scan delivered 0.15 s late as on the real lidar. Since the state on the circle is known, the steady-state force balance also gives the friction of the floor: on five floors from 0.90 to 1.10 times the tiles the estimate ended within 0.6 %.

On the car the drift never held for more than 0.5 s (Fig. 5). The rear let go only on a torque step at full lock, never under the rate-limited commands of the NMPC; once sliding, any correction, a little less steering or a little less speed, gripped it again within 0.25–0.5 s. Open loop, opening the steering from the donut at full throttle, the car keeps sliding only from 16°, on a 0.19 m circle; at 12–14° it fades back to grip. The tight stable drifts exist on the car

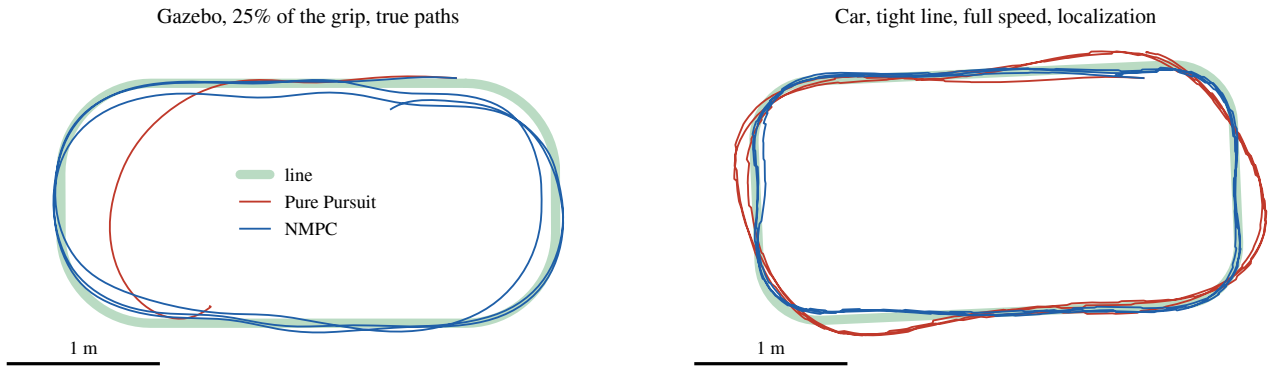


Fig. 4. Left: Gazebo at 25% of the grip of the tiles, true paths; Pure Pursuit spins in the first lap. Right: the car on the tight line at full speed, paths from its localization.

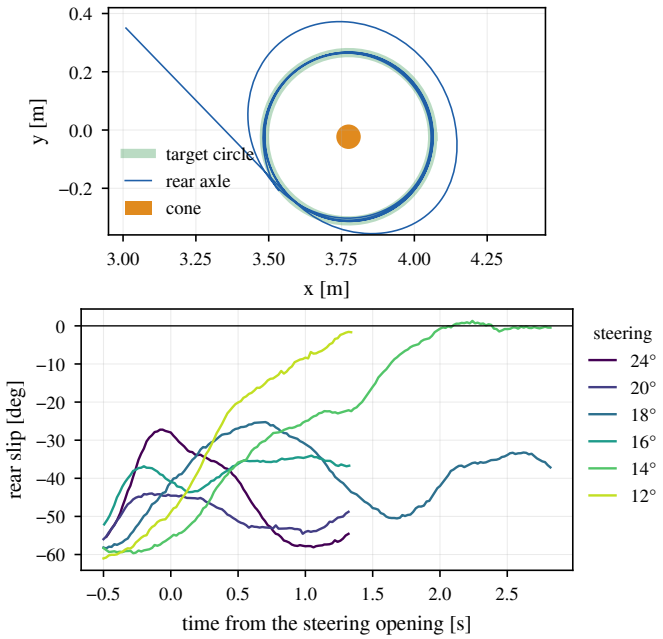


Fig. 5. Top: Gazebo, the drift NMPC around the cone, true path of the rear axle. Bottom: the car, open loop, rear slip after the steering opens from the donut at full throttle; it keeps sliding only from 16°.

as in the model; the unstable middle state the simulator held does not. The model's rear tire loses grip too easily at moderate longitudinal slip, which places the unstable equilibrium where the car has none. The work moved to the one regime where the model was demonstrably right: the full slide with the rear wheels locked.

VI. Drift parking

A. Handbrake turn

A first parking drove straight, turned, and at a set gyro heading commanded speed zero: the rear wheels lock and the car slides. Open loop the car stopped anywhere between 177° and 197°. In the slide the steering still acts, so every 20 ms the node runs the model to the stop for a constant steering and finds, by bisection, the one that stops the car at the heading asked; the locked rear slides with 1.5 times the friction of a spinning one. Steered, the stops were $(181.6 \pm 2.2)°$ over 28 runs. This parking ends nose first: its body sweeps 40–50 cm behind the stop and needs a gap of about 0.7 m.

B. Sideways into a gap

To park sideways into a tighter gap, the manoeuvre was designed in the refitted model first. Over 6000 random runs (a straight, one segment of steering and speed, a braked slide) and their local refinement, no run fits in a slot under 0.47 m, 1.8 car lengths: from 0.43 m/s, 0.93 s at full lock and full throttle, then braked.

The gap node follows that pattern (Fig. 6). At rest it finds the faces of the boxes as a line of lidar points on its left, and the gap as the break in that line, which also gives the heading to park at. It holds the line of the start at 0.4 m/s and kicks where the predicted stop leaves as much room at the nose as beyond the swept tail: full lock and full throttle, the rear steps out and the car drifts round at 3.4–4.0 rad/s. It then brakes when the gyro heading plus the turn of the slide after the brake reaches 180°,

$$\psi_{\text{brake}} + \Delta\psi_{\text{slide}} = 180°, \tag{5}$$

with $\Delta\psi_{\text{slide}} = 36.4°$ measured over the closed-loop runs. The gyro reading is 47 ms old when the node reads it, and the node carries it forward with the refitted model and sends the brake when it is due, between its 20 ms steps.

The brake still landed with a delay that varied from run to run: it acted 10–41 ms after the command, 24 ms on average, 2–8° of heading at these yaw rates. The cause was the firmware, which serviced the micro-ROS executor every 20 ms and took one queued /drive message per pass; serviced every 5 ms, the brake acts (10 ± 3) ms after the command.

Over the closed-loop runs from 0.60 m down to 0.47 m the car stopped at $(180.8 \pm 2.9)°$ (nine runs); Table III lists the clearances. What limits it now is the slide after

TABLE III
Gap parking on the tiles, full lock

gap	runs	stop [°]	closest, box A / B [cm]
0.60 m	6	171.4–183.8	7.1–14.2 / 4.0–9.2
0.54 m	5	172.2–183.3	2.7–10.9 / 6.4–11.6
0.50–0.52 m	7	176.5–185.5	1.9–6.7 / 2.9–8.1
0.47 m	9	173.2–185.7	2.5–6.1 / 1.7–6.8

the brake itself: 29–43° depending on the floor, 29.5° on freshly wiped tiles (the one touch), 35° or 42° on used ones, the two modes parting 100 ms after the brake. From the yaw rate, the rest of the turn is known to 1.1° only 150 ms into the slide, too late for the servo to correct it.

VII. Parking anywhere and the gymkhana

Parking anywhere. Two boxes are put down by hand anywhere in the mapped room. At rest, the lidar points more than 10 cm from the walls of the map are new; two groups of them whose sides towards the car lie on one line, 0.40–0.75 m apart along it, are the boxes and the gap. The parking starts on that line, where the gap node would start from rest. The approach is the shortest arc–straight–arc path [21] of 0.5 m radius that ends on 0.3 m of the line with the car's outline 5 cm clear of walls and boxes; when that start cannot be reached driving forward, or when it costs less (a metre backed up counts as two driven), the path ends further along the line and the gap node backs up first. The NMPC drives the approach (Fig. 7). Over 11 runs with the boxes at six places the car followed the plan within 1.4–4.8 cm rms and stopped between 175.2° and 191.4°; six runs backed up 0.7–2.5 m, and one touched a box.

The gymkhana. One launch from a start mark chains the nodes, each starting when the one before exits cleanly: the NMPC drives a lap with a slalom round two cones, the gap node a donut braked on the gyro to end at its starting heading (it then resets the localization, which the spinning wheels have confused), the NMPC drives back to the start, and the gap node parks. On the day of the video, under the camera, ten takes were clean[1]: 28.9–34.9 s each, the lap 3.3–5.6 cm rms from its path, the cones at least 8.1 cm from the body, parked at 179.3–182.3°, at least 5.4 cm from both boxes. Two earlier takes had stopped on their own: a new node on the Pi found the running ones 3–6 s late through DDS discovery, brought to 0.6–1.5 s by sending the discovery announcements more often.

VIII. Simulation twin

In Gazebo [22] a C++ system plugin replaces the ESP32 and the car. It publishes and subscribes the same topics and integrates the identified model at 1 kHz with the measured IMU noise and the brake lag of the motor driver; the room is built from the map and the lidar is modelled on the A1. The whole stack runs unchanged on simulated time. With Pure Pursuit it laps in 10.1 s as the car does, 4.4 cm rms from the line against 2.7 cm on the car: the model turns 4–6% less in steady corners. The drift replayed open loop gives the same donut with 6–9° more rear slip; the model has no motor torque limit. The twin carried the work the car could not do safely, friction as a parameter (Sec. IV), and the design and debugging of every node before it ran on the car, including the drift that the car then did not reproduce.

[1]One more was launched with the car still parked from the take before, not on its start mark, and drove into a box.

IX. Conclusion

A low-cost car on hard tires parks itself sideways into a gap of 1.8 car lengths, between boxes it finds itself, with every computation on board. Three lessons stand out. The model was good enough for an NMPC in grip and for locked-wheel slides, but not for the transition between grip and drift, and a controller that only works in simulation can rest on an equilibrium the real system does not have. Delays matter at these yaw rates: a firmware delay varying between 10 ms and 41 ms was worth several degrees at the stop, and every sensor needed its own time base. And the remaining spread of the parking, a slide that varies by ±3–4° with the state of the floor, is decided after the brake, where the car can no longer act on it. Natural next steps are an onboard estimate of the floor friction, which the drift experiments in simulation showed to be observable from the steering, a second surface, and a faster lidar.

References

[1] R. Y. Hindiyeh and J. C. Gerdes, "A controller framework for autonomous drifting: Design, stability, and experimental validation," Journal of Dynamic Systems, Measurement, and Control, vol. 136, no. 5, p. 051015, 2014.

[2] J. Y. Goh, T. Goel, and J. C. Gerdes, "Toward automated vehicle control beyond the stability limits: Drifting along a general path," Journal of Dynamic Systems, Measurement, and Control, vol. 142, no. 2, p. 021004, 2020.

[3] J. Z. Kolter, C. Plagemann, D. T. Jackson, A. Y. Ng, and S. Thrun, "A probabilistic approach to mixed open-loop and closed-loop control, with application to extreme autonomous driving," in IEEE International Conference on Robotics and Automation (ICRA), 2010, pp. 839–845.

[4] A. Liniger, A. Domahidi, and M. Morari, "Optimization-based autonomous racing of 1:43 scale RC cars," Optimal Control Applications and Methods, vol. 36, no. 5, pp. 628–647, 2015.

[5] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1TENTH: An open-source evaluation environment for continuous control and reinforcement learning," in Proceedings of the NeurIPS 2019 Competition and Demonstration Track, ser. Proceedings of Machine Learning Research, vol. 123, 2020, pp. 77–89.

[6] M. Cutler and J. P. How, "Autonomous drifting using simulation-aided reinforcement learning," in IEEE International Conference on Robotics and Automation (ICRA), 2016, pp. 5442–5448.

[7] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," Science Robotics, vol. 7, no. 66, p. eabm6074, 2022.

[8] T. Moore and D. Stouch, "A generalized extended Kalman filter implementation for the Robot Operating System," in Intelligent Autonomous Systems 13 (IAS-13), ser. Advances in Intelligent Systems and Computing, vol. 302. Springer, 2016, pp. 335–348.

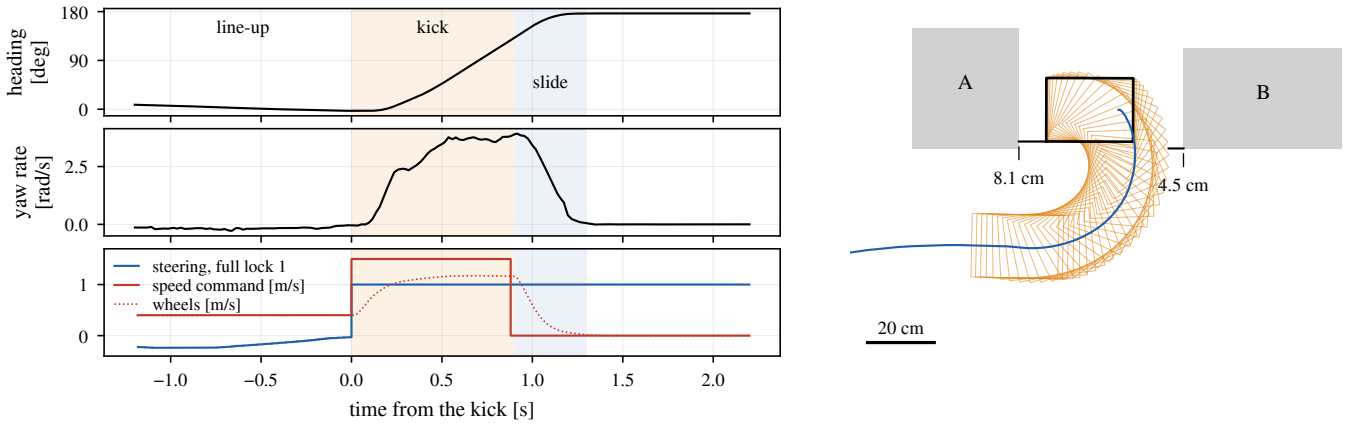


Fig. 6. The parking into the 47 cm gap of Fig. 1. Left: heading and yaw rate from the gyro, and the commands: the line-up at 0.4 m/s, the kick at full lock and full throttle, the brake, the slide. Right: the body every 1/30 s from the kick to the stop, the boxes where the lidar found them, and the closest approaches.

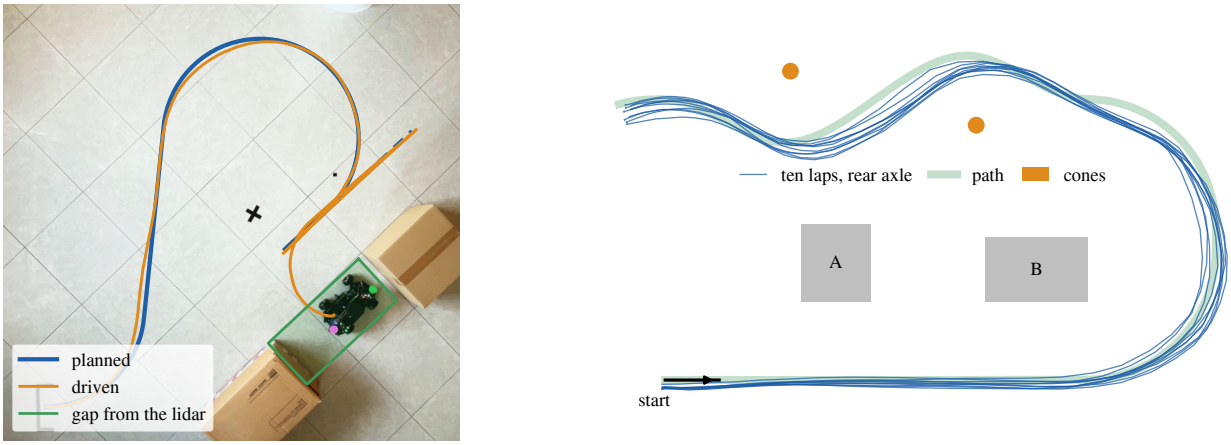


Fig. 7. Left: parking anywhere, the boxes at a third place; the path planned from the lidar (blue), which runs past the gap before the car backs up 0.9 m along it, and the path driven by the NMPC (orange), over the overhead camera at the stop. Right: the slalom lap of the ten gymkhana takes filmed for the video, rear axle paths from the lidar, with the cones and the boxes.

[9] D. Fox, W. Burgard, F. Dellaert, and S. Thrun, "Monte Carlo localization: Efficient position estimation for mobile robots," in Proceedings of the Sixteenth National Conference on Artificial Intelligence (AAAI), 1999, pp. 343–349.

[10] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," Journal of Open Source Software, vol. 6, no. 61, p. 2783, 2021.

[11] H. B. Pacejka, Tire and Vehicle Dynamics, 3rd ed. Butterworth-Heinemann, 2012.

[12] M. R. Jardin and E. R. Mueller, "Optimized measurements of unmanned-air-vehicle mass moment of inertia with a bifilar pendulum," Journal of Aircraft, vol. 46, no. 3, pp. 763–775, 2009.

[13] P. J. Besl and N. D. McKay, "A method for registration of 3-D shapes," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 14, no. 2, pp. 239–256, 1992.

[14] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, and M. Diehl, "acados—a modular open-source framework for fast embedded optimal control," Mathematical Programming Computation, vol. 14, no. 1, pp. 147–183, 2022.

[15] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi: a software framework for nonlinear optimization and optimal control," Mathematical Programming Computation, vol. 11, no. 1, pp. 1–36, 2019.

[16] E. Hairer and G. Wanner, Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems, 2nd ed. Springer, 1996.

[17] M. Diehl, H. G. Bock, and J. P. Schlöder, "A real-time iteration scheme for nonlinear optimization in optimal feedback control," SIAM Journal on Control and Optimization, vol. 43, no. 5, pp. 1714–1736, 2005.

[18] G. Frison and M. Diehl, "HPIPM: a high-performance quadratic programming framework for model predictive control," IFAC-PapersOnLine, vol. 53, no. 2, pp. 6563–6569, 2020.

[19] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Carnegie Mellon University, Robotics Institute, Tech. Rep. CMU-RI-TR-92-01, 1992.

[20] E. Velenis, E. Frazzoli, and P. Tsiotras, "Steady-state cornering equilibria and stabilisation for a vehicle during extreme operating conditions," International Journal of Vehicle Autonomous Systems, vol. 8, no. 2–4, pp. 217–241, 2010.

[21] L. E. Dubins, "On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents," American Journal of Mathematics, vol. 79, no. 3, pp. 497–516, 1957.

[22] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2004, pp. 2149–2154.